

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Solutions:

Q1. Testing as an Engineering Activity in a Banking Application

Software testing should be treated as an engineering activity because testing must be planned, systematic, measurable, and repeatable rather than being performed randomly. In the given banking application, frequent failures are occurring because the team follows unstructured testing practices. To improve the situation, the testing process should begin with proper requirement analysis. Testers should understand important banking functions such as login, account balance, fund transfer, withdrawal, deposit, transaction history, and payment processing. Based on these requirements, a detailed test plan should be prepared by defining the testing scope, resources, schedule, risks, tools, and responsibilities.

Systematic test design should then be introduced. Instead of selecting test cases randomly, testers can use techniques such as equivalence partitioning, boundary value analysis, decision table testing, state transition testing, and risk-based testing. For example, if a banking application allows a customer to transfer between ₹100 and ₹1,00,000, testers should test values below the minimum, at the minimum, normal values, at the maximum, and above the maximum. Both valid and invalid conditions should be included. Testing tools can also be introduced for automation, defect tracking, and test management. Standards and defined testing procedures should be followed so that every tester follows a common process.

For example, when testing a fund-transfer module, the tester should check valid transfers, insufficient balance, invalid account numbers, duplicate transfers, network interruption, transaction limits, and incorrect authentication. If a defect is found, it should be recorded with its steps to reproduce, expected result, actual result, severity, priority, and supporting evidence. After the defect is fixed, retesting and regression testing should be performed. This engineering-based approach improves defect detection, reduces repeated failures, provides better test coverage, and increases the reliability and quality of the banking application.

Q2. Structured Testing Process for an ATM System

A structured testing process is necessary for an ATM system because banking operations involve money and security, so even a small defect can cause serious problems. Testing should be performed throughout the Software Development Life Cycle rather than only after development is completed. During the requirement analysis stage, testers study requirements such as PIN validation, balance enquiry, cash withdrawal, deposit, transaction limits, receipt generation, and account updates. Any unclear or incomplete requirement should be identified at this stage. During test planning, the team defines the testing scope, test environment, resources, schedule, risks, and responsibilities.

During the test design stage, test cases are created for different ATM functions. For withdrawal, testers should prepare test cases for valid withdrawals, insufficient balance, invalid PIN, blocked card, zero withdrawal, negative values, maximum withdrawal limit, and network failure. During the test execution stage, the tester enters the required inputs and compares the actual output with the expected output. Any difference should be reported as a defect. After the developer fixes the defect, the tester performs retesting to verify the correction and regression testing to ensure that the correction has not affected other ATM functions.

For example, if a customer has ₹5,000 and attempts to withdraw ₹6,000, the ATM should reject the transaction and display an appropriate message. If the ATM deducts ₹6,000 despite insufficient balance, it is a serious defect. A structured testing process helps identify such problems systematically. It reduces missed test scenarios, improves consistency between testers, supports early defect detection, and increases the reliability and security of the ATM system.

Q3. Errors, Defects, and Failures in a Web Application

In software testing, error, defect, and failure are related but different concepts. An error is a human mistake made during requirements analysis, design, coding, or other development activities. A defect is the flaw introduced into the software because of that error. A failure occurs when the defect is executed and the software produces an incorrect result or behaves differently from the expected behavior. Understanding

these terms is important because improper understanding can lead to unclear defect reports and difficulty in finding the root cause.

For example, suppose a developer misunderstands the requirement that a password must contain at least eight characters. The developer writes code that accepts passwords with only four characters. The misunderstanding by the developer is the error, the incorrect validation logic in the application is the defect, and when a four-character password is accepted during execution, the incorrect behavior is the failure. Similarly, if a transaction is expected to display ₹5,000 but the application displays ₹50,000, the tester should clearly report the expected and actual results rather than simply writing "transaction failed."

A good defect report should contain a unique defect ID, clear title, module name, steps to reproduce, expected result, actual result, severity, priority, environment details, and screenshots or logs when required. Clear classification helps developers understand whether the issue is related to functionality, data, performance, security, or another area. Therefore, proper understanding of errors, defects, and failures improves communication between testers and developers, reduces debugging time, and improves the quality of the software.

Q4. Defect Clustering and Testing Principles

Defect clustering is an important software testing principle which states that a small number of modules usually contain a large number of defects. This means that defects are not always distributed equally throughout the application. Testers should therefore analyze previous defect data and identify modules that are more risky or contain a higher concentration of defects. Testing effort can then be prioritized toward these modules instead of spending equal effort on every part of the system.

Other testing principles should also guide test planning. Testing can show the presence of defects but cannot prove that the software is completely defect-free. Exhaustive testing is generally impossible because there are too many combinations of inputs and conditions. Therefore, testers should select important and high-risk scenarios. Early testing should be performed so that defects are identified before development progresses too far. The pesticide paradox states that if the same test cases are repeatedly used, they may stop finding new defects, so test cases should be regularly reviewed and improved.

For example, in an online banking system, the fund-transfer and payment modules may contain more defects because they involve complex calculations, database operations, authentication, and external services. Testers can therefore prioritize these modules and perform functional, integration, security, performance, and regression testing. Applying these principles helps use testing resources efficiently, detect important defects earlier, reduce production failures, and improve overall software quality.

Q5. Defect Classification and Defect Repository

A defect repository is a centralized collection of information about defects identified during software development and testing. It helps the organization maintain a history of defects and use previous defect information for future testing. Defects can originate from many sources, including incorrect requirements, design mistakes, programming errors, incorrect data, interface problems, configuration problems, and inadequate testing. Proper classification makes it easier to understand the nature and frequency of defects.

Defects can be classified into categories such as functional defects, performance defects, security defects, usability defects, compatibility defects, interface defects, and data-related defects. For example, if a payment is processed twice for a single request, it can be classified as a functional defect. If the application becomes extremely slow when many users access it, it can be classified as a performance defect. A defect repository should contain information such as defect ID, description, module, severity, priority, steps to reproduce, expected result, actual result, root cause, assigned developer, status, and resolution.

Repository data can be used to identify recurring problems. For example, if several releases contain duplicate-payment defects, testers can identify this pattern and create additional regression test cases for duplicate transactions. Developers can also investigate the root cause and improve coding or design practices. Therefore, effective defect classification and repository management help prevent recurring defects, improve test planning, increase traceability, and improve software quality across releases.

Q6. Mobile Banking Application Crashes During Fund Transfer

Frequent crashes during fund transfer in a mobile banking application may occur because of incorrect input handling, network failures, database problems, concurrency issues, memory problems, or incorrect transaction processing. To identify the root cause, testing should be performed systematically rather than only repeating the same normal transfer scenario. Testers should first study the requirements and identify all possible conditions under which fund transfer should work or fail.

Test cases should be prepared for valid transfers, insufficient balance, invalid account numbers, invalid amounts, maximum transaction limits, incorrect authentication, network disconnection, server failure, repeated transfer requests, and simultaneous transfers. For example, suppose a user initiates a ₹5,000 transfer and the network disconnects immediately after the sender's account is debited. The tester must verify whether the transaction is completed correctly and whether retrying the transaction causes a duplicate debit. Logs, database records, and server responses should be analyzed to identify the root cause.

Performance testing and stress testing should also be performed to determine whether the application crashes under heavy usage. Proper exception handling, input validation, transaction management, automated regression testing, and monitoring should be introduced. Systematic testing helps identify failures before release, reduces crashes, prevents duplicate transactions, and improves user confidence in the banking application.

Q7. Testing and Debugging

Testing and debugging are two different but related activities in software development. Testing is performed to identify defects or failures in the software, while debugging is performed to find the root cause of a defect and correct it. Testing generally involves executing the software with different inputs and comparing actual results with expected results. Debugging involves examining program logic, code, variables, logs, database operations, and execution flow to determine why the failure occurred.

For example, consider an ATM that deducts ₹5,000 when the customer requests ₹500. The tester identifies this incorrect behavior during testing and reports it as a defect. The developer then investigates the calculation logic and may discover that an incorrect variable or conversion has been used. The developer corrects the code and performs debugging. The tester then performs retesting to verify the fix and regression testing to ensure that other withdrawal functions have not been affected.

If developers confuse testing with debugging, defects may not be reported properly or testing may stop after the developer believes the code has been fixed. A structured approach should therefore be followed where testers identify and report failures, developers debug and correct the underlying defects, and testers verify the corrections. This separation improves defect detection, communication, accountability, and software quality.

Q8. Cross-Browser Testing in an E-Commerce Platform

Cross-browser testing is performed to ensure that a web application behaves correctly across different browsers, operating systems, and screen sizes. An e-commerce platform may behave differently in Chrome, Firefox, Edge, Safari, or different versions of these browsers because browsers may interpret HTML, CSS, JavaScript, and other technologies differently. Therefore, testing only one browser does not provide sufficient coverage.

A proper cross-browser testing strategy should first identify the browsers and devices commonly used by the application's customers. Testers should prepare test cases for important functions such as login, registration, product search, product details, shopping cart, checkout, payment, order confirmation, and order tracking. The same test cases should then be executed across the selected browser and operating-system

combinations. For example, if the checkout button works in Chrome but does not respond in Firefox, the tester should investigate whether the issue is related to JavaScript or browser compatibility.

Tools such as Selenium and Playwright can be used to automate repetitive browser tests. Responsive testing should also be performed for different screen sizes. Defects may originate from browser-specific JavaScript, CSS rendering, unsupported features, cookies, sessions, or browser configuration. Systematic cross-browser testing improves compatibility, reduces customer-facing failures, and provides a consistent user experience.

Q9. Defect Classification in a Payroll System

A payroll system must perform accurate calculations because incorrect salary calculations can directly affect employees. Logical errors in salary calculation can originate from incorrect requirements, design mistakes, coding errors, incorrect input data, or integration problems. Defect classification helps testers understand the type and source of the problem and select suitable testing techniques.

For example, suppose an employee's basic salary is ₹30,000 and the total deduction is ₹3,000. The expected net salary is ₹27,000. If the application subtracts the deduction twice and produces ₹24,000, the problem is a logical or coding defect. Testers should also check boundary conditions such as zero salary, maximum salary, minimum working days, maximum overtime, and tax limits. Decision tables can be used when salary calculations depend on multiple conditions such as attendance, tax category, overtime, allowances, and deductions.

Unit testing should be used to test individual salary formulas, while integration testing should verify the connection between attendance, employee, tax, and payroll modules. Regression testing should be performed after every correction. Proper requirement reviews, code reviews, automated calculations, and validation can prevent similar defects in future releases. This improves salary accuracy and reliability.

Q10. Structured Test Case Design

Random testing without proper test case design may fail to identify important defects because testers may concentrate only on common scenarios. Testing should instead be treated as a systematic engineering activity. The first step is to analyze the requirements and identify all functions and conditions that need to be tested. Test scenarios should then be created, followed by detailed test cases containing test data, expected results, and execution conditions.

Techniques such as equivalence partitioning, boundary value analysis, decision table testing, state transition testing, and risk-based testing can improve test effectiveness. For example, in a login system, testers should not only test valid username and password combinations. They should also test incorrect passwords, incorrect usernames, empty fields, locked accounts, special characters, maximum input lengths, and repeated failed attempts.

After execution, actual results should be compared with expected results. Defects should be documented clearly and assigned appropriate severity and priority. Once fixed, defects should be retested and regression tests should be executed. Structured test case design provides better coverage, reduces duplication, makes testing repeatable, and improves defect detection compared with random testing.

Q11. Data Inconsistency in a Healthcare Application

Data consistency is extremely important in a healthcare application because patient information must remain accurate across all modules. Data inconsistency may occur when one module contains different information from another module. Possible causes include incorrect database updates, integration defects, incorrect data mapping, duplicate records, failed transactions, or concurrency problems.

For example, a patient's prescription module may show a dosage of 500 mg while the pharmacy or billing module shows 250 mg. Such inconsistency can occur because data was incorrectly transferred between modules. Testers should verify the complete data flow between patient registration, prescription, laboratory, pharmacy, and billing modules. Database testing should be performed to verify that the correct values are stored and retrieved. Integration testing should ensure that information transferred between modules remains accurate.

Defects should be recorded in a centralized defect repository with clear descriptions, reproduction steps, expected results, actual results, severity, and supporting evidence. Regression testing should be performed after corrections. Data validation rules, database constraints, transaction management, audit logs, and proper integration testing can help prevent recurring problems. This systematic approach improves data integrity and reliability.

Q12. Repeated Defects and Defect Repository

A company may continue to experience repeated defects even though testing is performed because previous defect information is not being effectively analyzed. A defect repository can help solve this problem by maintaining historical information about defects found in different releases. Testers can use this information to identify patterns, high-risk modules, and recurring root causes.

For example, if three different releases contain defects related to duplicate transactions, the repository can reveal that similar validation problems have occurred repeatedly. Testers can analyze the root cause and create permanent regression test cases for duplicate transactions. They can also increase testing around modules that frequently produce defects. Defects should be categorized based on module, type, severity, priority, root cause, and release.

The repository should not only be used for recording defects but also for learning from previous failures. Regular defect trend analysis, root-cause analysis, regression test creation, and knowledge sharing should be performed. This allows the organization to move from simply finding defects to preventing them. As a result, recurring defects are reduced, testing becomes more effective, and software quality improves over time.

Q13. Web-Based Learning System During Peak Usage

A web-based learning system may work correctly under normal conditions but fail when thousands of users access it simultaneously. This indicates that peak usage scenarios were not adequately considered during testing. Functional testing alone cannot guarantee that the application will remain reliable under heavy load. Therefore, performance and load testing should be included in the test plan.

Testers should identify the expected number of concurrent users and critical operations such as login, course access, video streaming, examination, assignment submission, and result viewing. Load testing should simulate the expected number of users, while stress testing should gradually increase the load beyond the expected capacity to determine the breaking point. Endurance testing can be used to identify problems that occur during long periods of continuous usage.

For example, if 5,000 students attempt an online examination at the same time, testers should verify whether users can log in, load questions, submit answers, and receive confirmation without excessive delays or failures. Server CPU, memory, database performance, network usage, and response time should be monitored. Better test planning and performance coverage can identify capacity problems before deployment and improve system reliability.

Q14. Early Tester Involvement in SDLC

Testers should be involved from the beginning of the Software Development Life Cycle because early testing helps identify defects before they become expensive and difficult to fix. If testers participate only after development is completed, requirement misunderstandings and design problems may remain unnoticed for a long time.

During requirement analysis, testers can review requirements and identify ambiguous, incomplete, or conflicting conditions. During design, they can review system architecture and identify possible integration and usability risks. During development, testers can prepare test cases and test data in parallel with coding. For example, in an ATM system, early tester involvement may identify requirements for invalid PIN attempts, withdrawal limits, insufficient balance, card blocking, and transaction cancellation before coding begins.

Early collaboration between developers, testers, business analysts, and other stakeholders improves communication and reduces misunderstandings. It also supports early defect detection and reduces the cost of fixing problems. Therefore, testers should participate in requirement reviews, design discussions, sprint planning, and defect analysis meetings. This results in better quality and more reliable software.

Q15. Incomplete Test Coverage

Incomplete test coverage occurs when certain parts of the software or important input conditions are not tested. A system may appear to work correctly during normal situations but fail when unusual or boundary conditions occur. Therefore, testers must identify coverage gaps during test planning and design appropriate test cases.

For example, consider an ATM withdrawal function. If the tester checks only a normal withdrawal of ₹1,000, important conditions such as zero amount, negative amount, insufficient balance, minimum withdrawal, maximum withdrawal, and withdrawal above the daily limit may remain untested. Equivalence partitioning can divide inputs into valid and invalid groups, while boundary value analysis can test values around minimum and maximum limits.

Requirement traceability can be used to ensure that each requirement has corresponding test cases. Code coverage techniques such as statement and branch coverage can also identify untested areas when applicable. Regression testing should ensure that previously working functionality continues to work after changes. Better coverage improves defect detection, reduces unexpected failures, and increases confidence in software reliability.

Q16. Defect Reporting in a Logistics Application

Effective defect reporting is important because developers depend on tester reports to understand and reproduce problems. Poor reporting practices can cause delays because developers may not have enough information to reproduce the defect. A structured defect report should contain a unique ID, clear title, module, environment, steps to reproduce, expected result, actual result, severity, priority, and relevant screenshots or logs.

For example, instead of reporting "delivery tracking is wrong," the tester should specify the shipment number or test data used, the steps followed, the expected tracking status, and the actual status displayed by the system. This allows the developer to reproduce the issue quickly. Severity indicates the technical or business impact of the defect, while priority indicates how urgently it should be fixed.

A standardized defect reporting format should be followed by all testers. Defects should be stored in a centralized tracking system and updated as they move through states such as new, assigned, fixed, retested, reopened, and closed. Clear reporting improves communication, reduces resolution time, prevents duplicate reports, and makes defect analysis easier.

Q17. Process-Based Testing Across Modules

Process-based testing ensures that a common and standardized testing procedure is followed across all modules of a software system. If each tester follows a different process, some modules may receive extensive testing while others may receive insufficient testing. This can lead to inconsistent quality and missed defects.

The organization should define common procedures for requirement analysis, test planning, test case design, test execution, defect reporting, retesting, regression testing, and test closure. Standard templates should be used for test cases and defect reports. Entry and exit criteria should also be defined so that testers know when testing can begin and when it can be considered complete.

For example, in an e-commerce application, the login, product search, shopping cart, payment, and order tracking modules should all follow the same basic testing process. Risk-based prioritization can then be used to give additional attention to critical modules such as payment. Standardized testing improves consistency, traceability, communication, test coverage, and overall software quality.

Q18. Boundary Testing and Edge Cases

Edge cases are situations that occur at the extreme limits of input or system conditions. They are often missed when testers use only normal test data. Boundary Value Analysis is an effective technique for identifying such cases because defects frequently occur at the boundaries of input ranges.

For example, suppose an ATM allows a withdrawal between ₹100 and ₹10,000. Testers should test ₹99, ₹100, ₹101, ₹9,999, ₹10,000, and ₹10,001. ₹99 and ₹10,001 should be rejected, while ₹100 and ₹10,000 should be accepted if all other conditions are satisfied. Testing values immediately before, at, and after the boundaries helps identify incorrect comparison operators and off-by-one errors.

Boundary test cases should be included in automated regression suites so that they are not forgotten in future releases. Testers should also consider zero values, negative values, empty inputs, maximum lengths, and unusually large values wherever applicable. Applying boundary testing improves input validation, detects hidden defects, and reduces failures after deployment.

Q19. Defect Repository and Documentation

A defect repository is a centralized system used to document and track software defects throughout the development and testing process. Without proper documentation, organizations may lose information about previous defects, making it difficult to identify recurring problems. Proper documentation provides a history of what went wrong, how it was fixed, and what testing should be performed in the future.

A good defect record should contain a defect ID, title, description, affected module, steps to reproduce, expected result, actual result, severity, priority, environment, assigned developer, status, root cause, and resolution. For example, if a logistics application incorrectly updates an order status, the tester should document the exact steps and test data required to reproduce the issue.

The repository can also support analysis by showing which modules contain the most defects, which types of defects occur frequently, and which problems repeat across releases. This information can be used to improve test planning and create additional regression tests. Proper defect documentation improves communication, traceability, root-cause analysis, and prevention of recurring defects.

Q20. Banking Transaction Processing

Repeated failures in a banking transaction system indicate that the existing testing approach may not be systematic or comprehensive. Since transaction processing involves financial data, testing should cover functional correctness, data integrity, security, performance, and error handling. The process should begin with requirement analysis, followed by test planning and structured test case design.

Testers should prepare cases for deposits, withdrawals, fund transfers, incorrect account numbers, insufficient balances, transaction limits, duplicate requests, network failures, database failures, and simultaneous transactions. For example, if a customer transfers ₹2,000 from Account A to Account B, Account A should be reduced by exactly ₹2,000 and Account B should increase by exactly ₹2,000. If the network fails during the operation, the system should maintain transaction consistency and should not process the same transfer twice.

Integration testing should verify communication between the banking application, database, authentication system, and external payment services. Performance and stress testing should determine how the system

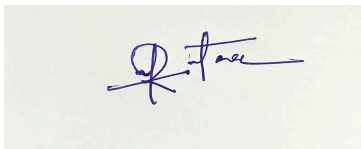
behaves under heavy transaction loads. Security testing should verify authentication and authorization. After defects are fixed, retesting and regression testing should be performed. A structured testing approach improves transaction accuracy, prevents recurring failures, improves security, and increases customer trust in the banking system.

Code of Conduct:

I Prartana T(192421099) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand